Submitted by:

Shehriyar khan

Submitted to:

Ma'am Zeenat Zulfiqar

Course title:

Artificial Intelligence

Registration no:

SP23-BSE-162

Deadline:

14 April 2026

BFS LAB TASK ACTIVITY 01:

Table of contents

BFS CODE

+ Section

[2]
0s

```python
graph = {
 '5' : ['3','7'],
 '3' : ['2', '4'],
 '7' : ['8'],
 '2' : [],
 '4' : ['8'],
 '8' : []
}
##The above code will create a graph with the entries given in the figure above.

visited = [] # List for visited nodes.
queue = [] #Initialize a queue

def bfs(visited, graph, node): #function for BFS
 visited.append(node)
 queue.append(node)
 while queue: # Creating loop to visit each node
  m = queue.pop(0)
  print (m, end = " ")
  for neighbour in graph[m]:
   if neighbour not in visited:
    visited.append(neighbour)
    queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, '5') # function calling
```

```
Following is the Breadth-First Search
5 3 7 2 4 8
```

ACTIVITY 02:

Table of contents

BFS CODE

+ Section

BFS CODE

[4]

```
# python program to demonstrate some of the above
# terminologies
# Function to add an edge between vertices x and y
# Function to print the parent of each node
def printParents(node, adj, parent):
    # current node is Root, thus, has no parent
    if (parent == 0):
        print(node, "->Root")
    else:
        print(node, "->", parent)

    # Using DFS
    for cur in adj[node]:
        if (cur != parent):
            printParents(cur, adj, node)
# Function to print the children of each node
def printChildren(Root, adj):
    # Queue for the BFS
    q = []

    # pushing the root
    q.append(Root)

    # visit array to keep track of nodes that have been
    # visited
    vis = [0]*len(adj)

    # BFS
    while (len(q) > 0):
        node = q[0]
        q.pop(0)
        vis[node] = 1
        print(node, "-> ", end=" ")
        for cur in adj[node]:
            if (vis[cur] == 0):
                print(cur, " ", end=" ")
                q.append(cur)
        print("\n")
# Function to print the leaf nodes
def printLeafNodes(Root, adj):
    # Leaf nodes have only one edge and are not the root
    for i in range(0, len(adj)):
        if (len(adj[i]) == 1 and i != Root):
            print(i, end=" ")
    print("\n")
# Function to print the degrees of each node
def printDegrees(Root, adj):
    for i in range(1, len(adj)):
        print(i, ": ", end=" ")
```

```
        # Root has no parent, thus, its degree is equal to
        # the edges it is connected to
        if (i == Root):
            print(len(adj[i]))
        else:
            print(len(adj[i])-1)
# Driver code
# Number of nodes
N = 7
Root = 1
# Adjacency list to store the tree
adj = []
for i in range(0, N+1):
    adj.append([])

# Creating the tree
adj[1].append(2)
adj[2].append(1)
adj[1].append(3)
adj[3].append(1)
adj[1].append(4)
adj[4].append(1)
adj[2].append(5)
adj[5].append(2)
adj[2].append(6)
adj[6].append(2)
adj[4].append(7)
adj[7].append(4)
# Printing the parents of each node
print("The parents of each node are:")
printParents(Root, adj, 0)
# Printing the children of each node
print("The children of each node are:")
printChildren(Root, adj)
# Printing the leaf nodes in the tree
print("The leaf nodes of the tree are:")
printLeafNodes(Root, adj)
# Printing the degrees of each node
print("The degrees of each node are:")
printDegrees(Root, adj)
```

OUTPUT:

```
The parents of each node are:
1 ->Root
2 -> 1
5 -> 2
6 -> 2
3 -> 1
4 -> 1
7 -> 4
The children of each node are:
1 ->  2   3   4

2 ->  5   6

3 ->

4 ->  7

5 ->

6 ->

7 ->

The leaf nodes of the tree are:
3 5 6 7

The degrees of each node are:
1 :  3
2 :  2
3 :  0
4 :  1
5 :  0
6 :  0
7 :  0
```

Start coding or generate with AI.